



Name: Fatima Asif

Roll No: 231980015

Course : Big Data Analytics

Assignment NO 2

Instructor: Sir Zuhaib Hussain Butt

Part 1– Environment Setup

- Python Version
- Spark Version
- Operating System

```
... Python Version: 3.12.13 (main, Mar  4 2026, 09:23:07) [GCC 11.4.0]
    Operating System: Linux
    Spark Version: 4.0.3
```

- Java Version

```
!java -version

... openjdk version "17.0.19" 2026-04-21
    OpenJDK Runtime Environment (build 17.0.19+10-1-22.04.2-Ubuntu)
    OpenJDK 64-Bit Server VM (build 17.0.19+10-1-22.04.2-Ubuntu, mixed mode, sharing)
```

- Spark configuration

```
... Spark Configuration
spark.rdd.compress: True
spark.hadoop.fs.s3a.vectored.read.min.seek.size: 128K
spark.app.startTime: 1785675883155
spark.executor.extraJavaOptions: -Djava.net.preferIPv6Addresses=false -XX:+Ignore
spark.sql.artifact.isolation.enabled: false
spark.sql.warehouse.dir: file:/content/spark-warehouse
spark.master: local[*]
spark.app.id: local-1785675886194
spark.app.submitTime: 1785675881838
spark.driver.port: 33967
spark.driver.host: a66e949e2fb0
spark.executor.id: driver
spark.submit.pyFiles:
spark.driver.extraJavaOptions: -Djava.net.preferIPv6Addresses=false -XX:+Ignor
spark.hadoop.fs.s3a.vectored.read.max.merged.size: 2M
spark.submit.deployMode: client
spark.serializer.objectStreamReset: 100
spark.app.name: NYC Taxi Analytics
spark.ui.showConsoleProgress: true
```

Part 2– Load Dataset

Dataset Description: This dataset is the NYC Yellow Taxi Trip Records (January 2024) dataset and is in the Parquet format. This includes data about the trips made by taxis which have the time when the pickup occurred, time when the drop-off occurred, number of passengers, trip distance, payment amount, mode of payment, and location of pickup and drop-off.

```
+ Code + Text
df = spark.read.parquet("yellow_tripdata_2024-01.parquet")
```

- Schema

```
df.printSchema()

... root
  |-- VendorID: integer (nullable = true)
  |-- tpep_pickup_datetime: timestamp_ntz (nullable = true)
  |-- tpep_dropoff_datetime: timestamp_ntz (nullable = true)
  |-- passenger_count: long (nullable = true)
  |-- trip_distance: double (nullable = true)
  |-- RatecodeID: long (nullable = true)
  |-- store_and_fwd_flag: string (nullable = true)
  |-- PULocationID: integer (nullable = true)
  |-- DOLocationID: integer (nullable = true)
  |-- payment_type: long (nullable = true)
  |-- fare_amount: double (nullable = true)
  |-- extra: double (nullable = true)
  |-- mtaxi_taxi: double (nullable = true)
  |-- tip_amount: double (nullable = true)
  |-- tolls_amount: double (nullable = true)
  |-- improvement_surcharge: double (nullable = true)
  |-- total_amount: double (nullable = true)
  |-- congestion_surcharge: double (nullable = true)
  |-- Airport_fee: double (nullable = true)
```

- Number of columns

```
print("Number of Columns:", len(df.columns))
```

... Number of Columns: 19

- Number of records

```
print("Number of Records:", df.count())
```

Number of Records: 2754895

- First 20 rows

```
df.show(20, truncate=False)
```

	VendorID	trip_pickup_datetime	trip_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID	passenger_rating
1	2	2024-01-01 00:57:55	2024-01-01 01:17:43	1	11.72	1	N	186	79	2
1	1	2024-01-01 00:03:00	2024-01-01 00:09:36	1	11.8	1	N	140	236	1
1	1	2024-01-01 00:17:06	2024-01-01 00:35:01	1	4.7	1	N	236	79	1
1	1	2024-01-01 00:36:38	2024-01-01 00:44:56	1	1.4	1	N	79	211	1
1	1	2024-01-01 00:46:51	2024-01-01 00:52:57	1	0.8	1	N	211	148	1
1	1	2024-01-01 00:54:08	2024-01-01 01:26:31	1	14.7	1	N	148	141	1
1	2	2024-01-01 00:49:44	2024-01-01 01:15:47	2	10.82	1	N	138	181	1
1	1	2024-01-01 00:30:40	2024-01-01 00:58:40	0	3.0	1	N	246	231	2
1	2	2024-01-01 00:26:01	2024-01-01 00:54:12	1	5.44	1	N	161	261	2
1	2	2024-01-01 00:28:08	2024-01-01 00:29:16	1	0.04	1	N	113	113	2
1	2	2024-01-01 00:35:22	2024-01-01 00:41:41	2	0.75	1	N	107	137	1
1	1	2024-01-01 00:25:00	2024-01-01 00:34:03	2	11.2	1	N	158	246	1
1	1	2024-01-01 00:35:16	2024-01-01 01:11:52	2	8.2	1	N	246	190	1
1	1	2024-01-01 00:43:27	2024-01-01 00:47:11	2	0.4	1	N	68	90	1
1	1	2024-01-01 00:51:53	2024-01-01 00:55:43	1	0.8	1	N	90	68	2
1	1	2024-01-01 00:50:09	2024-01-01 01:03:57	1	5.0	1	N	132	216	2
1	1	2024-01-01 00:41:06	2024-01-01 00:53:42	1	11.5	1	N	164	79	1
1	2	2024-01-01 00:52:09	2024-01-01 00:52:28	1	0.0	1	N	237	237	2
1	2	2024-01-01 00:56:38	2024-01-01 01:03:17	1	11.5	1	N	141	263	1
1	2	2024-01-01 00:32:34	2024-01-01 00:49:33	1	12.57	1	N	161	263	1

only showing top 20 rows

Part 3– Exploratory Data Analysis

1. Total trips

```
... Total Trips: 2964624
```

2. Earliest trip date

```
... +-----+
| Earliest_Trip_Date|
+-----+
|2002-12-31 22:59:39|
+-----+
```

3. Latest trip date

```
... +-----+
| Latest_Trip_Date|
+-----+
|2024-02-01 00:01:15|
+-----+
```

4. Unique vendors

```

+-----+
|VendorID|
+-----+
|      1|
|      2|
|      6|
+-----+

Number of Unique Vendors: 3

```

5. Average trip distance

```

... +-----+
    |avg(trip_distance)|
    +-----+
    |3.6521691789580624|
    +-----+

```

6. Average fare

```

... +-----+
    | avg(fare_amount)|
    +-----+
    |18.175061916792536|
    +-----+

```

7. Maximum fare

```

... +-----+
    |max(fare_amount)|
    +-----+
    |          5000.0|
    +-----+

```

8. Minimum fare

```

+-----+
|min(fare_amount)|
+-----+
|          -899.0|
+-----+

```

9. Average passenger count

```

... +-----+
    |avg(passenger_count)|
    +-----+
    |  1.3392808966805005|
    +-----+

```

10. Number of payment method

```

... +-----+
    |payment_type|
    +-----+
    |      1|
    |      3|
    |      2|
    |      4|
    |      0|
    +-----+

Number of Payment Methods: 5

```

Part 4– Data Cleaning

- Original Records:

```

... Original Number of Records: 2964624

```

- Remove duplicates

```
Number of Records After Removing Duplicates:
2964624
```

Explanation: We use `dropDuplicates()` to remove duplicate rows from the dataset. It ensures that each trip record must be one time.

- Remove invalid trip distances

```
... Records After Removing Invalid Trip Distances:
2904253
```

Explanation: It removes records where the trip distance is zero or negative. These records may consider invalid for analysis

- Remove negative fares

Records After Removing Negative Fares:
2870188

Explanation: It removes records with negative fare values. Fare amounts cannot normally be negative.

Missing Values:

```

... -RECORD 0-----
VendorID              | 0
tpep_pickup_datetime  | 115293
tpep_dropoff_datetime | 115293
passenger_count        | 115293
trip_distance          | 0
RatecodeID            | 115293
store_and_fwd_flag     | 115293
PULocationID          | 0
DOLocationID          | 0
payment_type           | 0
fare_amount            | 0
extra                  | 0
mta_tax                | 0
tip_amount             | 0
tolls_amount           | 0
improvement_surcharge | 0
total_amount           | 0
congestion_surcharge   | 115293
Airport_fee            | 115293

```

Explanation: This displays the number of missing values in each column.

```
Records After Removing Missing Values:
2754895
```

```

... Final Number of Records: 2754895
-----
| vendorID | trip_pickup_datetime | trip_dropoff_datetime | passenger_count | trip_distance | RatecodeID | store_and_fwd_flag | PULocationID | DOLocationID | passenger_count |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 1 | 2024-01-01 00:47:21 | 2024-01-01 00:51:58 | 2 | 1.13 | 1 | IN | 238 | 166 | 1 |
| 1 | 2024-01-01 00:41:15 | 2024-01-01 00:45:01 | 3 | 0.8 | 1 | IN | 114 | 234 | 2 |
| 2 | 2024-01-01 00:42:51 | 2024-01-01 01:16:30 | 1 | 4.43 | 1 | IN | 144 | 142 | 1 |
| 2 | 2024-01-01 00:29:42 | 2024-01-01 00:43:10 | 1 | 3.64 | 1 | IN | 41 | 142 | 2 |
| 2 | 2024-01-01 00:40:27 | 2024-01-01 01:04:32 | 2 | 5.9 | 1 | IN | 114 | 256 | 1 |
| 2 | 2024-01-01 00:50:36 | 2024-01-01 01:17:49 | 2 | 3.43 | 1 | IN | 90 | 141 | 1 |
| 2 | 2024-01-01 00:27:14 | 2024-01-01 00:34:39 | 1 | 0.95 | 1 | IN | 234 | 107 | 1 |
| 2 | 2024-01-01 00:12:32 | 2024-01-01 00:20:12 | 1 | 1.83 | 1 | IN | 234 | 107 | 1 |
| 2 | 2024-01-01 00:55:46 | 2024-01-01 01:08:28 | 1 | 2.25 | 1 | IN | 144 | 170 | 1 |
| 1 | 2024-01-01 00:04:46 | 2024-01-01 00:09:25 | 2 | 0.8 | 1 | IN | 43 | 239 | 1 |
-----
only showing top 10 rows

```

Explanation:

Missing values were identified by counting null values in each column, and rows containing missing values were removed using `dropna()`. These steps ensured that the dataset was consistent, accurate, and suitable for further analysis.

Part 5– Spark Transformations

1. filter()

Purpose: Filter trips with a fare greater than \$50

Code:

```
high_fare = df.filter(df.fare_amount > 50)
high_fare.show(10)
```

VendorID	trip_pickup_datetime	trip_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID	passenger_fare
2	2024-01-01 01:41:09	2024-01-01 02:07:52	1	19.69	5	n	262	265	68
1	2024-01-01 05:38:26	2024-01-01 06:13:34	1	18.2	2	n	132	68	132
2	2024-01-01 05:40:30	2024-01-01 06:11:23	1	19.41	2	n	88	132	265
2	2024-01-01 06:49:16	2024-01-01 07:08:22	1	14.35	1	n	132	265	132
2	2024-01-01 07:19:04	2024-01-01 07:52:13	2	19.35	2	n	132	166	263
2	2024-01-01 07:10:11	2024-01-01 07:40:35	5	19.95	2	n	132	263	185
2	2024-01-01 08:44:01	2024-01-01 09:14:55	1	18.23	1	n	132	185	132
2	2024-01-01 09:37:07	2024-01-01 10:02:16	1	20.38	2	n	140	132	107
2	2024-01-01 10:29:35	2024-01-01 10:36:03	2	3.88	2	n	145	107	25
2	2024-01-01 11:53:23	2024-01-01 12:22:10	1	18.4	1	n	132	25	

only showing top 10 rows

2. select()

Purpose: Select only required columns

Code:

```
selected = df.select("VendorID", "passenger_count", "trip_distance", "fare_amount")
selected.show(10)
```

VendorID	passenger_count	trip_distance	fare_amount
2	2	1.13	7.9
1	3	0.8	6.5
2	1	4.43	31.0
2	1	3.64	17.7
2	2	5.9	30.3
2	2	3.43	24.7
2	1	0.95	8.6
2	2	1.83	10.7
2	1	2.25	14.2
1	2	0.8	7.2

only showing top 10 rows

3. withColumn()

Purpose: Create a new column that calculates fare per mile.

```
from pyspark.sql.functions import round

fare_per_mile = df.withColumn(
    "fare_per_mile",
    round(df.fare_amount / df.trip_distance, 2)
)

fare_per_mile.show(10)
```

```
***
```

VendorID	trip_pickup_datetime	trip_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID	payment_type	fare_amount
2	2024-01-01 00:47:21	2024-01-01 00:51:58	2	1.13	1	N	238	166	1	166
1	2024-01-01 00:41:15	2024-01-01 00:45:01	3	0.8	1	N	114	234	2	234
2	2024-01-01 00:42:51	2024-01-01 01:10:30	1	4.43	1	N	144	142	1	142
2	2024-01-01 00:29:42	2024-01-01 00:43:10	1	3.64	1	N	41	142	2	142
2	2024-01-01 00:40:27	2024-01-01 01:04:32	2	5.9	1	N	114	256	1	256
2	2024-01-01 00:50:36	2024-01-01 01:17:49	2	3.43	1	N	90	141	1	141
2	2024-01-01 00:27:14	2024-01-01 00:34:39	1	0.95	1	N	234	107	1	107
2	2024-01-01 00:12:32	2024-01-01 00:20:12	2	1.83	1	N	236	237	1	237
2	2024-01-01 00:55:46	2024-01-01 01:08:28	1	2.25	1	N	144	170	1	170
1	2024-01-01 00:04:46	2024-01-01 00:09:25	2	0.8	1	N	43	239	1	239

only showing top 10 rows

4. orderBy()

Purpose: Sort trips by fare amount in descending order.

```
sorted_df = df.orderBy(df.fare_amount.desc())
sorted_df.show(10)
```

VendorID	trip_pickup_datetime	trip_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID	payment_type	fare_amount
2	2024-01-14 10:08:11	2024-01-16 13:54:22	1	31.95	1	N	220	220	1	220
2	2024-01-02 07:50:00	2024-01-02 11:20:20	1	233.25	4	N	168	265	1	265
2	2024-01-06 21:01:38	2024-01-06 23:41:44	1	142.62	4	N	132	265	1	265
2	2024-01-22 16:40:43	2024-01-22 19:11:20	1	157.25	5	N	265	265	1	265
2	2024-01-01 21:59:33	2024-01-01 21:59:55	4	0.21	5	N	132	132	1	132
2	2024-01-17 10:46:15	2024-01-17 13:48:11	1	109.75	4	N	161	265	1	265
2	2024-01-14 15:40:26	2024-01-14 18:00:27	1	122.47	4	N	132	265	1	265
2	2024-01-26 23:59:09	2024-01-27 02:45:39	1	120.76	4	N	143	265	1	265
2	2024-01-28 17:24:07	2024-01-28 19:50:30	1	119.46	4	N	132	265	1	265
2	2024-01-07 12:38:39	2024-01-07 12:38:59	4	0.11	5	N	246	265	1	265

only showing top 10 rows

5. drop()

Purpose: Remove the **store_and_fwd_flag** column

```
dropped = df.drop("store_and_fwd_flag")
dropped.show(10)
```

VendorID	trip_pickup_datetime	trip_dropoff_datetime	passenger_count	trip_distance	RatecodeID	PULocationID	DOLocationID	payment_type	fare_amount
2	2024-01-01 00:47:21	2024-01-01 00:51:58	2	1.13	1	238	166	1	166
1	2024-01-01 00:41:15	2024-01-01 00:45:01	3	0.8	1	114	234	2	234
2	2024-01-01 00:42:51	2024-01-01 01:10:30	1	4.43	1	144	142	1	142
2	2024-01-01 00:29:42	2024-01-01 00:43:10	1	3.64	1	41	142	2	142
2	2024-01-01 00:40:27	2024-01-01 01:04:32	2	5.9	1	114	256	1	256
2	2024-01-01 00:50:36	2024-01-01 01:17:49	2	3.43	1	90	141	1	141
2	2024-01-01 00:27:14	2024-01-01 00:34:39	1	0.95	1	234	107	1	107
2	2024-01-01 00:12:32	2024-01-01 00:20:12	2	1.83	1	236	237	1	237
2	2024-01-01 00:55:46	2024-01-01 01:08:28	1	2.25	1	144	170	1	170
1	2024-01-01 00:04:46	2024-01-01 00:09:25	2	0.8	1	43	239	1	239

only showing top 10 rows

6. distinct()

Purpose: Display unique payment methods.

Code:

```
payment_methods = df.select("payment_type").distinct()
payment_methods.show()
```

payment_type
1
3
2
4

7. groupBy()

Purpose: Calculate the average fare for each payment type.

```
from pyspark.sql.functions import avg
grouped = df.groupBy("payment_type").agg(
    avg("fare_amount").alias("Average_Fare")
)
grouped.show()
```

payment_type	Average_Fare
1	18.376087258999238
3	17.040781294018178
2	18.613623047913922
4	19.66531404344601

8. join()

Purpose: Join the grouped data with unique payment types

```
payment_df = df.select("payment_type").distinct()

joined = payment_df.join(
    grouped,
    on="payment_type",
    how="inner"
)

joined.show()
```

```
... +-----+-----+
|payment_type|Average_Fare|
+-----+-----+
|1|18.376087258999238|
|3|17.040781294018178|
|2|18.613623047913922|
|4|19.66531404344601|
+-----+-----+
```

9. alias()

Purpose: Rename the columns in the output

```
alias_df = df.select(
    df.VendorID.alias("Vendor"),
    df.trip_distance.alias("Distance"),
    df.fare_amount.alias("Fare")
)

alias_df.show(10)
```

```
... +-----+-----+-----+
|Vendor|Distance|Fare|
+-----+-----+-----+
|2|1.13|7.9|
|1|0.8|6.5|
|2|4.43|31.0|
|2|3.64|17.7|
|2|5.9|30.3|
|2|3.43|24.7|
|2|0.95|8.6|
|2|1.83|10.7|
|2|2.25|14.2|
|1|0.8|7.2|
+-----+-----+-----+
only showing top 10 rows
```

10. repartition()

Purpose: Repartition the dataset into 4 partition to improve parallel processing.

```
repartitioned = df.repartition(4)

print("Partitions:", repartitioned.rdd.getNumPartitions())

Partitions: 4
```

Part 6– Spark SQL

Step 1: Create a Temporary View

```
df.createOrReplaceTempView("taxi_trips")
```

Query 1: Top 10 longest trips

```
... +-----+-----+-----+
|trip_distance|fare_amount|passenger_count|
+-----+-----+-----+
|15400.32|28.9|1|
|10879.28|70.0|1|
|1715.22|70.0|1|
|971.8|21.5|1|
|964.6|39.5|1|
|277.4|33.8|2|
|246.22|8.6|1|
|233.25|1616.5|1|
|210.82|500.0|1|
|210.2|650.0|1|
+-----+-----+-----+
```

Query 2: Top Pickups Locations

PULocationID	Total_Trips
132	138130
237	137093
161	136500
236	129604
162	102308
186	100737
230	100008
142	98906
138	87253
239	82391

Query 3: Average Fare by payment type

payment_type	Average_Fare
1	18.38
2	18.61
3	17.04
4	19.67

Query 4: Peak Pickup Hour

Pickup_Hour	Total_Trips
18	198037

Query 5: Trips Over 20 Miles

VendorID	trip_distance	fare_amount
2	15400.32	28.9
2	10879.28	70.0
2	1715.22	70.0
1	971.8	21.5
1	964.6	39.5
2	277.4	33.8
2	246.22	8.6
2	233.25	1616.5
2	210.82	500.0
1	210.2	650.0

only showing top 10 rows

Query 6: Monthly Revenue

Month	Revenue
1	7.542614842E7
2	90.47
12	235.12

Query 7: Average Trip Distance By vendor

VendorID	Avg_Distance
1	3.13
2	3.35

Query 8: Highest Fare Trips

```
*** +-----+ +-----+ +-----+
| VendorID | fare_amount |
+-----+ +-----+ +-----+
| 2 | 2221.3 |
| 2 | 1616.5 |
| 2 | 912.3 |
| 2 | 899.0 |
| 2 | 820.0 |
| 2 | 761.1 |
| 2 | 749.2 |
| 2 | 744.3 |
| 2 | 739.4 |
| 2 | 700.0 |
+-----+ +-----+ +-----+
```

Query 9: Average Passenger Count

```
*** +-----+
| Average_Passengers |
+-----+
| 1.34 |
+-----+
```

Query 10: Payment Method Usage

```
*** +-----+ +-----+
| payment_type | Total_Trips |
+-----+ +-----+
| 1 | 2298422 |
| 2 | 422945 |
| 4 | 22879 |
| 3 | 10649 |
+-----+ +-----+
```

Part 7– Window Functions

- row number()

Purpose: Assign a unique row number to each trip based on descending fare amount.

```
*** +-----+ +-----+ +-----+
| VendorID | fare_amount | row_number |
+-----+ +-----+ +-----+
| 2 | 2221.3 | 1 |
| 2 | 1616.5 | 2 |
| 2 | 912.3 | 3 |
| 2 | 899.0 | 4 |
| 2 | 820.0 | 5 |
| 2 | 761.1 | 6 |
| 2 | 749.2 | 7 |
| 2 | 744.3 | 8 |
| 2 | 739.4 | 9 |
| 2 | 700.0 | 10 |
+-----+ +-----+ +-----+
only showing top 10 rows
```

- rank()

Purpose: Rank trips by fare amount. If two trips have the same fare, they receive the same rank, and the next rank is skipped.

```
*** +-----+ +-----+ +-----+
| VendorID | fare_amount | rank |
+-----+ +-----+ +-----+
| 2 | 2221.3 | 1 |
| 2 | 1616.5 | 2 |
| 2 | 912.3 | 3 |
| 2 | 899.0 | 4 |
| 2 | 820.0 | 5 |
| 2 | 761.1 | 6 |
| 2 | 749.2 | 7 |
| 2 | 744.3 | 8 |
| 2 | 739.4 | 9 |
| 2 | 700.0 | 10 |
+-----+ +-----+ +-----+
only showing top 10 rows
```

- dense rank()

Purpose: Rank trips by fare amount without skipping rank numbers when there are ties.

VendorID	fare_amount	dense_rank
2	2221.3	1
2	1616.5	2
2	912.3	3
2	899.0	4
2	820.0	5
2	761.1	6
2	749.2	7
2	744.3	8
2	739.4	9
2	700.0	10

only showing top 10 rows

Part 8– Performance Optimization

Measure Execution Time Before Caching

```
***
```

payment_type	count
1	2298422
3	10649
2	422945
4	22879

Execution Time Before Caching: 19.392117500305176 seconds

Cache the Dataset

```
print('Dataset Caching Status:')
*** Dataset cached successfully!
```

Measure Execution Time After Caching

```
***
```

payment_type	count
1	2298422
3	10649
2	422945
4	22879

Execution Time After Caching: 4.79041314125061 seconds

Demonstrate repartition()

```
print("Partitions Before:", df.rdd.getNumPartitions())

repartitioned_df = df.repartition(4)

print("Partitions After:", repartitioned_df.rdd.getNumPartitions())

Partitions Before: 200
Partitions After: 4
```

Show the Execution Plan with explain()

```
df.groupby("payment_type").count().explain(True)

== Parsed Logical Plan ==
Aggregate [payment_type, [payment_type, 'count(1) AS count#3208]]
+- Project [coalesce(VendorID#0, cast(0.0 as int)) AS VendorID#592, tpep_pickup_datetime#1, tpep_dropoff_datetime#2, coalesce(passenger_count, 0) AS passenger_count#31, trip_distance#4, Ratecode#0]
+- Filter (fare_amount#10 >= cast(0 as double))
+- Filter (trip_distance#4 > cast(0 as double))
+- Deduplicate [DOLocationID#8, improvement_surcharge#15, tpep_dropoff_datetime#2, PULocationID#7, tolls_amount#14, trip_amount#14]
+- Relation [VendorID#0,tpep_pickup_datetime#1,tpep_dropoff_datetime#2,passenger_count#31,trip_distance#4,RatecodeID#51,store_fare_amount#10,trip_distance#4]

== Analyzed Logical Plan ==
payment_type: bigint, count: bigint
Aggregate [payment_type#598L], [payment_type#598L, count(1) AS count#3208L]
+- Project [coalesce(VendorID#0, cast(0.0 as int)) AS VendorID#592, tpep_pickup_datetime#1, tpep_dropoff_datetime#2, coalesce(passenger_count, 0) AS passenger_count#31, trip_distance#4, Ratecode#0]
+- Filter (fare_amount#10 >= cast(0 as double))
+- Filter (trip_distance#4 > cast(0 as double))
+- Deduplicate [DOLocationID#8, improvement_surcharge#15, tpep_dropoff_datetime#2, PULocationID#7, tolls_amount#14, trip_amount#14]
+- Relation [VendorID#0,tpep_pickup_datetime#1,tpep_dropoff_datetime#2,passenger_count#31,trip_distance#4,RatecodeID#51,store_fare_amount#10,trip_distance#4]

== Optimized Logical Plan ==
Aggregate [payment_type#598L], [payment_type#598L, count(1) AS count#3208L]
+- Project [payment_type#598L]
+- IndexSpoolRelation [VendorID#592, tpep_pickup_datetime#1, tpep_dropoff_datetime#2, passenger_count#593L, trip_distance#594, RatecodeID#595]
+- AdaptiveSparkPlan isFinalPlan=true
```

Part 9– Spark UI Analysis

Jobs

[YARN HDFS1 | Jupyter](#)
[NYC Taxi Analytics Application](#)
[Send chat message](#)
[Load spark-submit](#)
[Load spark-submit](#)
[Load spark-submit](#)
[Load spark-submit](#)
[Load spark-submit](#)
[Load spark-submit](#)
[Load spark-submit](#)
[Load spark-submit](#)
[Load spark-submit](#)
[Load spark-submit](#)
[Load spark-submit](#)
[Load spark-submit](#)
[Load spark-submit](#)
[Load spark-submit](#)
[Load spark-submit](#)
[Load spark-submit](#)
[Load spark-submit](#)
[Load spark-submit](#)

[delegator wasp: freedom.ngrok-free.dev/jobs/](#)
[Ask Gemini](#)
[Home](#)
[Search](#)
[Bookmarks](#)
[All Bookmarks](#)

[Spark Jobs](#)
[Stages](#)
[Storage](#)
[Environment](#)
[Executors](#)
[SQL / Data/rate](#)
[NYC Taxi Analytics application UI](#)

Spark Jobs ^(?)

User: root

Started At: 2026/09/02 13:04:43

Total Uptime: 28 min

Scheduling Mode: FIFO

Completed Jobs: 122

Failed Jobs: 2

- Event Timeline
- Completed Jobs (122)**

Page: 1 2 > 2 Pages jump to 1 Show 100 items in a page Go

Job ID	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
123	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/09/02 13:18:19	5 s	1/1 (1 skipped)	200/200 (2 skipped)
122	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/09/02 13:16:19	0.2 s	1/1 (1 skipped)	1/1 (1 skipped)
121	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/09/02 13:16:11	7 s	1/1 (1 skipped)	200/200 (2 skipped)

Number of Jobs

Completed Jobs: 122

Failed Jobs: 2

Stages

The screenshot shows the Spark web console interface. At the top, there's a navigation bar with tabs for 'Stages', 'Storage', 'Environment', 'Executors', and 'SQL / DataFrame'. The 'Stages' tab is active. Below the navigation bar, the title 'Stages for All Jobs' is displayed. A summary section shows: 'Completed Stages: 125', 'Skipped Stages: 96', and 'Failed Stages: 7'. Below this, a link for 'Completed Stages (125)' is shown. The main content area displays a table of job stages. The table has columns for 'Stage ID', 'Description', 'Submitted', 'Duration', 'Tasks Succeeded/Total', 'Input', 'Output', 'Shuffle Read', and 'Shuffle Write'. The first four rows of the table are visible, showing stages 222, 220, 217, and 215. Each row includes a link to view details for that stage. The 'Tasks Succeeded/Total' column shows progress bars and values like '200/200' or '1/1'. The 'Input' and 'Output' columns show data sizes in MB or KiB. The 'Shuffle Read' and 'Shuffle Write' columns show shuffle data sizes in KiB.

Stage ID	Description	Submitted	Duration	Tasks Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
222	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 + details	2026/08/02 13:18:19	5 s	200/200	269.8 MiB			
220	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 + details	2026/08/02 13:18:19	0.2 s	1/1		49.2 KiB		
217	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 + details	2026/08/02 13:18:11	7 s	200/200	269.8 MiB			49.2 KiB
215	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 + details	2026/08/02 13:18:11	59 ms	1/1		11.5 KiB		

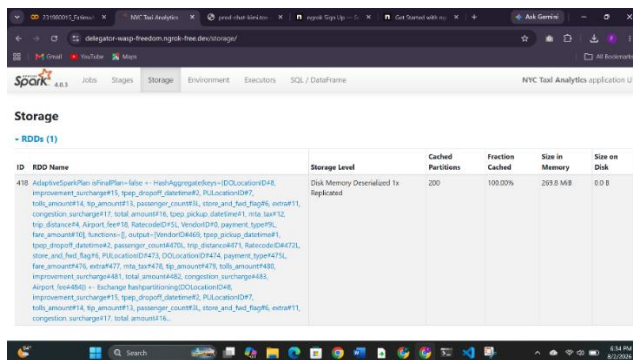
Number of Stages:

Completed Stages: 125

Skipped :96

Failed: 2

Storage



ID	RDD Name	Storage Level	Cached Partitions	Fraction Cached	Size in Memory	Size on Disk
418	AdaptiveSparkPlan (final) = false <- hashAggregate(keys=[IDLocationID418, improvement_surcharge#15, type_dropoff_datetime#2, PULocationID#7, toll_amount#14, tip_amount#13, passenger_count#3, store_and_fwd_flag#6, amount#1], output=[VendorID#60], top_pickup_datetime#1, mta_tax#12, tip_distance#4, Airport_fee#18, RatecodeID#25, VendorID#5, payment_type#9, fare_amount#10, function_5_output=[VendorID#60], top_pickup_datetime#1, type_dropoff_datetime#2, passenger_count#400, trip_distance#71, RatecodeID#4723, store_and_fwd_flag#6, PULocationID#423, OULocationID#44, payment_type#479, fare_amount#479, amount#7, mta_tax#12, tip_amount#479, toll_amount#480, improvement_surcharge#481, total_amount#482, congestion_surcharge#483, Airport_fee#480] > > Exchange Repartitioning(OULocationID#4, improvement_surcharge#15, type_dropoff_datetime#2, PULocationID#7, toll_amount#14, tip_amount#13, passenger_count#3, store_and_fwd_flag#6, amount#1, congestion_surcharge#17, total_amount#16...	Disk Memory Deserialized 1x Replicated	200	100.00%	293.8 MiB	0.0 B

Which operation caused shuffle? Why?

Shuffle involves redistribution of data into different partitions by Spark. In the case of the `groupBy()` function, Spark will have to redistribute data because it groups records having similar keys (`payment_type`). In the same way, ordering of data using the `orderBy()` function will involve redistribution of data by Spark to achieve global sorting. Due to data shuffling, shuffle is among the costliest operations in Spark.

Reflection

This assignment helped me understand how to use Apache Spark through Google Colab. I was able to understand how to load datasets, clean data, transform through Apache Spark, run Spark SQL commands, use window functions, cache data, and repartition datasets for optimal performance. Also, I gained knowledge on the usage of the Spark User Interface using ngrok to understand Spark job and stage execution.

The most challenging thing about this assignment was the fact that the Spark UI could not be accessed directly through Google Colab. However, configuring ngrok and linking it with the Spark UI helped a lot.

I also understood the difference between Spark and Pandas. Pandas is ideal for working with smaller datasets, whereas Spark is meant for larger datasets and processes them in parallel way, thus making it more efficient for handling big data. The other very important concept that I have understood is that of Lazy Evaluation, where Spark does not execute the code until and unless an action such as `count()` or `show()` is invoked.

In conclusion, this assignment improved my knowledge regarding Apache Spark and also its significance in the field of Big Data analysis.